DM548: Computer Architecture & System Programming
DM588: Computer Architecture

Fall 2026

User-Defined Data Types

- Enums
- Structures
- Unions
- Bit Fields
- Type Alias

Enums, Fancy `int` Constants

```
enum MyEnum { // Named
    A, B, C
};
enum { // Unnamed
    D, E = 42, F
};
void doStuff(enum MyEnum mode);
```

- Declares a new type, with an underlying type which is `int`.
- So, `enum MyEnum` is a distinct type, which is implemented as an `int`.
- The `enum` keyword is part of the name (more later).
- The enumerators are **not** scoped inside the enum, but are in global scope. They are `int` constants.
- Enumerators can be given a constant value, or will get one automatically.

---

https://en.cppreference.com/w/c/language/enum

- Enums
- Structures
- Unions
- Bit Fields
- Type Alias

Structures

A sequence of variables of any type that will be stored together.
(The basics of a class, but it may be best to temporarily forget everything you know about Java/Python classes)

Examples (definition):

```
struct Point {
    int x;
    int y;
};
```

```
struct Employee {
    char name[256];
    int year, month, day;
};
```

Examples (declaration):

```
struct Point;
struct Employee;
```

Again, `struct` is actually part of the name of the new types.

---

https://en.cppreference.com/w/c/language/struct

Structures

- The variables within structs are called members.
- Variables of struct types can be declared as normal:
  ```
  struct Point p;
  ```
- As with (almost) any other variable, there is no initialisation.
- Members can be referred to with the dot operator:
  ```
  p.x = p.y = 42;
  ```
- Aggregate initialisation:
  ```
  struct Point p = {42, 60};
  struct Point p = {.y = 42};
  ```
- As other types, the total size can be found:
  ```
  sizeof(struct Point)
  ```
- The size may be larger than the sum of sizes of the members (padding may be inserted due to alignment).
- They can be copied: `struct Point q = p;`
- They can be returned from functions (i.e., a copy).
- They can be given as function arguments (i.e., a copy).

---

https://en.cppreference.com/w/c/language/struct_initialization

Copying Example

```c
#include <stdio.h>

struct A {
    int *p;
};

int main(void) {
    int i = 42;

    struct A a = {.p = &i};
    struct A b = a;
    *b.p = 60;
    printf("i = %d, *a.p = %d, *b.p = %d\n", i, *a.p, *b.p);
    return 0;
}
```

- Pointers are just addresses.
- They can be copied as any other variable.
- What they point to is not touched.

More Struct Examples

```
struct Triangle {
    struct Point a, b, c;
};
```

```
struct ListNode {
    int data;
    struct ListNode *next;
};
```

```
struct TreeNode {
    int data;
    struct TreeNode *parent;
    struct TreeNode *left, *right;
};
```

```
void foo(void) {
    struct Triangle t;
    t.a.x = 0;
    t.a.y = 42;
}
```

Member Access Through Pointers

It is common to have a pointer to some struct, which means member access would be: `(*p).x`
I.e., dereference `p`, then access the member `x`.
Shorthand Syntax: `p->x`

Example:

```
struct TreeNode {
    int data;
    struct TreeNode *parent;
    struct TreeNode *left, *right;
};

void Tree_unlinkSubtree(TreeNode *node) {
    if(!node->parent) return;
    if(node->parent->left == node)
        node->parent->left = NULL;
    else
        node->parent->right = NULL;
}
```

E.g., compare to "`(*(*node).parent).left == node`"

Array of Structures

- Structs can be array members as any other type:
  `struct Point points[42];`
- Array initialisation:
  `int arr[4] = {0, 20, 10, 2};`
  `int arr[4] = {[2] = 42, 43};`
  (makes 0, 0, 42, 43)
- Recall struct initialisation:
  `struct Point p = {42};` implicit 0 for y
  `struct Point q = {.y = 60};` implicit 0 for x
- They combine, recursively:

```
struct Triangle ts[4] = {
   {{.y = 100}, .c = {1, 2}} // first triangle
   {{2, 3}, {4, 5}, {6, 7}} // second triangle
   // all 0s for the rest
};
```

Structure Size and Alignment

Padding is inserted in structs to satisfy alignment requirements:

```c
struct A {
    char c; // size 1
    // probably size 3 padding
    int i; // probably size 4
};
```

```c
struct B {
    int i; // probably size 4
    char c; // size 1
    // probably size 3 padding
};
```

- The alignment of a variable is a power of 2.
- **alignof** (macro expanding to **_Alignof**) (C11)
  Query the alignment requirement of a type.
- **alignas** (macro expanding to **_Alignas**) (C11)
  Dictate the alignment for a variable (number or another type).

```c
alignas(double) int i;
alignas(64) int i;
```

Useful when explicitly using SSE, or for controlling false sharing in cache lines.

---

https://en.cppreference.com/w/c/language/_Alignof

https://en.cppreference.com/w/c/language/_Alignas

- Enums
- Structures
- Unions
- Bit Fields
- Type Alias

Unions

- In a struct, the members are disjoint in memory and can be set independently (they are also called product types).
- In a union, the members overlap and only one of them may be used at any one time (they are also called sum types).
- Unions are declared exactly like structs, except the keyword.

```c
union Data {
    int i;
    float f;
    char *s;
};

union Data d;
d.i = 42; // i is now active
d.f = 3.14; // switch to f
d.s = "Hello"; // switch to s
```

https://en.cppreference.com/w/c/language/union

Unions

- The size of a union is the size of the largest element.
- The alignment of a union is the largest alignment among the elements.
- Which member is active is not tracked by the compiler, nor at runtime.
- Reading through a member which is not active has undefined behaviour.

```
union {
    int i; float f;
} data;

data.i = 42;
int j = data.i; // ok, i is active
float g = data.f; // UB, f is not active
```

Unions

Common pattern: a union + an enum ("tagged union")

```
struct Data {
    enum { DataKind_INT, DataKind_FLOAT} kind;
    union {
        int i;
        float f;
    } data;
};
```

- Enums
- Structures
- Unions
- Bit Fields
- Type Alias

Bit Fields

- A bit field is an integer type with a given width. E.g., `unsigned int tag : 3;`
- Can only be declared inside structs and unions.
- Adjacent bit fields may share and straddle the underlying bytes.
- A bit field is either unsigned, signed, or have implementation-defined signedness.
- It is not possible to form pointers to bit fields, and they have no alignment.

---

https://en.cppreference.com/w/c/language/bit_field

- Enums
- Structures
- Unions
- Bit Fields
- Type Alias

Type Alias

- A type alias is created simply from a normal declaration, but with `typedef` in front.
- Variable declaration:

  ```
  int *arr[42];
  ```

  a variable **arr** which is an array of 42 pointers to `int`.
- Type alias declaration:

  ```
  typedef int *arr[42];
  ```

  a type alias **arr** for an array of 42 pointers to `int`.
- It's an alias, not a new type.
- Commonly used to alias the tagged names:

  ```
  typedef struct MyStruct MyStruct;
  typedef enum MyEnum MyEnum;
  typedef union MyUnion MyUnion;
  ```
- Also commonly used to alias function pointer types (more later), and in general complex types.

---

https://en.cppreference.com/w/c/language/typedef

Type Alias Example

```
// forward-declare and alias types
typedef struct Tree Tree;
typedef struct TreeNode TreeNode;

// define types
struct Tree  {
   // note: we can now write 'TreeNode'
   //       instead of 'struct TreeNode'
   TreeNode *root;
   int size;
};

struct TreeNode {
   int data;
   // note: we can now write 'TreeNode'
   //       instead of 'struct TreeNode'
   TreeNode *parent;
   TreeNode *left, *right;
};
```